ISM2411 — Lab Week 07

Functions + Debug First, Then Ask — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 07 · Unit 2 · Control Flow & Structure

Contents

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 07 Lab — Functions + Debug First, Then Ask
Unit	Unit 2 · Control Flow & Structure
Class length	75 minutes
Format	Live code-along, with one structured 20-minute debugging-protocol block (Exercise 5) run individually/in pairs on a timer
Prerequisites	Modules 05–06: `if`/`elif`/`else`, `for`/`while` loops, the accumulator pattern
Student-facing lab page	markumreed.github.io/ism2411 — week07_lab
Exercises covered	Exercises 1–6 (required) + Stretch 1/2 (as time allows)
Submission	`functions.py` + the Canvas reflection

Two genuinely distinct teaching goals live in this one lab. The first half (Exercises 1–4, 6) is standard technical content: `def`, parameters, `return`, and scope — the single biggest new syntax jump since `if`/`elif`/`else`. The second half (Exercise 5) is an **AI-literacy exercise**, not a programming exercise — a structured protocol for debugging *before* reaching for an AI assistant, and for using AI narrowly (to explain an error, not to hand over a fix) when you do reach for it. Run Exercise 5 on an actual timer, out loud, since its four steps are the entire point — a rushed or skipped version of it defeats the exercise.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Define a function with `def`, including parameters and a `return` statement, and explain the difference between a function *returning* a value and merely *printing* one.
2. Call one function from inside another (composition), and trace the flow of a value through a short chain of function calls.
3. Use a `for` loop to call the same function repeatedly over a list of inputs.
4. Explain local vs. global scope: why a variable assigned inside a function doesn't affect a same-named variable outside it.
5. Apply a disciplined, four-step debugging protocol — solo trace, narrowly-scoped AI question, self-written fix, debrief — before defaulting to "ask AI to fix it."

3 Before Class — Setup Checklist

- ☐ Open `functions.py`, empty except the header comment.
- ☐ Prepare `broken_sales.py` for Exercise 5 — if your course's Canvas page already provides this file, use that version instead of this guide's; if not, Appendix B below contains a verified three-bug version (one `SyntaxError`, one silent logic bug, one `IndexError`) ready to distribute or project.
- ☐ Have a real timer visible to the room for Exercise 5's four five-minute steps — a phone timer projected or a visible classroom clock works; the timing discipline is part of what makes the exercise work as a habit-forming ritual rather than a one-off.
- ☐ Decide your policy on AI tool access during Exercise 5's "AI Explainer" step (which specific tool, whether it's already permitted by your syllabus) and state it explicitly at the start of the exercise, rather than leaving it ambiguous in the moment.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+
- Students: `functions.py`, `broken_sales.py` (provided), a phone or water bottle for rubber-duck debugging (only half-joking — see Exercise 5), access to an AI assistant for the narrowly-scoped step only

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: functions as "package it once, call it by name"	4
0:04–0:10	Exercise 1 — `calculate_tax`	6
0:10–0:18	Exercise 2 — `apply_discount`	8
0:18–0:26	Exercise 3 — Compose them	8
0:26–0:34	Exercise 4 — Loop + function	8
0:34–0:54	Exercise 5 — Debug First, Then Ask (four 5-minute timed steps)	20
0:54–1:02	Exercise 6 — Scope experiment	8
1:02–1:10	Stretch 1 — Summarize a list with a function	8

Time	Segment	Minutes
1:10–1:15	Stretch 2 preview + wrap-up, reflection, submission checklist	5

Exercise 5 is deliberately allotted its full, literal 20 minutes (the lab page's own four 5-minute steps) — do not compress it to make room elsewhere; it is arguably the most important exercise of the semester for long-term habit formation, more than any single syntax lesson.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

> *"Every module so far, when you needed the same logic twice — like the discount tiers you wrote in Modules 5 and 6 — you retyped it, or copy-pasted it. Today you learn to package logic up* once*, give it a name, and call it by that name as many times as you want. That's a function. And the second half of today is different from anything we've done — a structured protocol for debugging your own code* before *asking an AI to fix it for you. Both halves matter equally today."*

Do: Open `functions.py`, type the header:

```
# functions.py
# ISM2411 Module 07 Lab — Functions + Debug First, Then Ask
```

6.2 Exercise 1 — `calculate_tax` (0:04–0:10, 6 min)

Teaching goal: The absolute minimum function: `def`, parameters, `return` — and the crucial distinction between a function that *returns* a value and one that merely prints.

Say to the class:

> *"Simplest possible function: takes a price and a rate, returns the tax. One line of logic — but the* syntax *around that one line is entirely new today."*

Live-code this:

```
# --- Exercise 1 ---
def calculate_tax(price, rate):
    return price * rate

print(f"Tax on $100 at 7%: ${calculate_tax(100, 0.07):.2f}")
print(f"Tax on $250 at 8%: ${calculate_tax(250, 0.08):.2f}")
```

Line-by-line explanation:

- `def calculate_tax(price, rate):` — `def` begins a function definition. `calculate_tax` is the name you're choosing — same naming rules as variables. `(price, rate)` are **parameters**: named placeholders for values the function expects to receive when called; they don't hold any actual value yet, they're just names the function body will refer to. The colon and the indented block below it work exactly like `if` and `for` — indentation marks what's "inside"

the function.

- `return price * rate` — **this is the line that makes it a function that produces a usable value**, rather than one that just prints. `return` hands a value back to whatever called the function, and immediately ends the function's execution — say explicitly: **nothing runs after a `return` statement inside the same function call.**
- `calculate_tax(100, 0.07)` — this is a **call**: the function actually runs now, with `price` bound to `100` and `rate` bound to `0.07` for the duration of this call only. The `return`ed value (`7.0`) is what this expression evaluates to — it slots directly into the f-string's `{...}`, formatted with `:.2f`.
- **The distinction to land explicitly:** `calculate_tax` never calls `print()` itself — it hands a number back. It's the *caller* (the `print(f"...")` line) that decides to display it. This matters because a function that returns a value can be *reused* in a calculation (like Exercise 3 is about to do), while a function that only prints cannot — you can't do math with what `print()` "gives back" (it gives back nothing usable, technically `None`).

Run it. Expected output:

```
Tax on $100 at 7%: $7.00
Tax on $250 at 8%: $20.00
```

Common student mistakes to watch for:

- Writing `print(price * rate)` inside the function instead of `return price * rate` — runs without error, and even *looks* right if you only run it once, but the function then hands back `None` to any code trying to use its result, which breaks Exercise 3's composition immediately. Demonstrate this live: replace `return` with `print`, then try `calculate_tax(100, 0.07) + 5` — worth showing the resulting `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'` as a preview of exactly the kind of bug this distinction prevents.
- Forgetting the parentheses when *calling* the function (writing `calculate_tax` instead of `calculate_tax(100, 0.07)`) — this doesn't error, but refers to the function *object itself*, not a call to it, and prints something like `<function calculate_tax at 0x...>` instead of a number. Worth a quick live look if it comes up, since the output is a strange enough thing to see that it's genuinely confusing without an explanation.

Check for understanding: "If I call `calculate_tax(250, 0.08)` five times in a row in my script, how many times do I have to write the formula `price * rate`?" (Once — inside the function definition. This is the entire value proposition of a function, worth stating explicitly as the payoff for the new syntax overhead.)

6.3 Exercise 2 — `apply_discount` (0:10–0:18, 8 min)

Teaching goal: A function whose body contains a full `if/elif/else` chain (Module 05's logic, now packaged) — confirms functions can contain any code students already know how to write, not just a single expression.

Say to the class:

> *"Functions aren't limited to one line. This one has a whole discount-tier decision inside it — the exact `if/elif/else` shape from Module 05, just packaged with a name and parameters now."*

Live-code this:

```
# --- Exercise 2 ---
def apply_discount(price, tier):
    if tier == "gold":
        discount = 0.15
    elif tier == "silver":
        discount = 0.10
    elif tier == "bronze":
        discount = 0.05
    else:
        discount = 0
    return price * (1 - discount)

for tier in ["gold", "silver", "bronze", "none"]:
    print(f"{tier}: ${apply_discount(200, tier):.2f}")
```

Line-by-line explanation:

- The `if/elif/else` block is entirely inside the function body — every line is indented one level under `def`, and the `if/elif` bodies are indented a second level under that, exactly the nested-indentation pattern from Module 06's Exercise 4.
- `return price * (1 - discount)` — placed *after* the entire `if/elif/else` chain, at the function's top indentation level (not inside any branch) — this is the crucial structural point: **all four branches lead to the same single `return` line**, rather than each branch having its own separate `return`. Ask the room: "would it work just as well to put `return price * (1 - discount)` inside each of the four branches instead, four separate times?" (Yes, functionally — but it's worth explicitly contrasting the two styles: one `return` after the decision is made is less repetitive and keeps the "what does this function give back" logic in one place, which is a real readability win as functions grow more complex.)
- The `for tier in [...]:` loop calls `apply_discount` four times, once per tier name in the list — direct rehearsal of Module 06's "loop + function-shaped logic" pattern, now with an actual function being called instead of inline code.

Run it. Expected output:

```
gold: $170.00
silver: $180.00
bronze: $190.00
none: $200.00
```

Common student mistakes to watch for:

- Indenting `return` one level too deep (inside the `else` block only) — this means every tier *except* `"none"` returns `None` instead of a discounted price, since only the `else` branch actually reaches a `return` statement; the other three branches fall off the end of the function with nothing returned. A good live demo, since the failure (three `None`s, one correct price) is visually striking.
- Misspelling a tier string (e.g. `"Gold"` with a capital G) when calling — string comparison is case-sensitive, so `"Gold" == "gold"` is `False`, silently falling through to the `else` branch and returning the undiscounted price with no error at all. This is squarely in the "silent wrong answer" category this course keeps returning to.

Check for understanding: "What does `apply_discount(200, "platinum")` return — a tier that doesn't exist in the function at all?" (`$200.00` — the full, undiscounted price, since `"platinum"` matches none of the `if`/`elif` conditions and falls through to `else`. No error, no warning — worth stating explicitly as a real design tradeoff: this function silently treats *any* unrecognized tier as "no discount," which may or may not be the desired behavior for unexpected input.)

6.4 Exercise 3 — Compose Them (0:18–0:26, 8 min)

Teaching goal: Call one function from inside the body of another — **composition** — and see directly why Exercise 1's choice of `return` over `print` was essential to making this possible at all.

Say to the class:

> *"This is the payoff for using `return` instead of `print` in both functions so far — we're about to feed one function's output directly into another, which is only possible because each one hands back a usable value."*

Live-code this, in two stages — first manually, then wrapped in a third function:

```
# --- Exercise 3 (stage 1: manual composition) ---
after_discount = apply_discount(200, "gold")
after_tax = after_discount + calculate_tax(after_discount, 0.07)
print(f"After gold discount: ${after_discount:.2f}")
print(f"After tax: ${after_tax:.2f}")
```

```
# --- Exercise 3 (stage 2: wrap it in a third function) ---
def final_price(price, tier, tax_rate):
    discounted = apply_discount(price, tier)
    return discounted + calculate_tax(discounted, tax_rate)

print(f'final_price(200, "gold", 0.07) = ${final_price(200, "gold", 0.07):.2f}')
```

Line-by-line explanation:

- `after_discount = apply_discount(200, "gold")` — Exercise 2's function, called and its return value stored in a variable, exactly like calling any built-in function (`float()`, `type()`) from prior modules — there's no special syntax difference between calling a function you wrote and one Python provides.
- `calculate_tax(after_discount, 0.07)` — note this passes `after_discount` (the *discounted* price, `$170.00`), not the original `$200` — tax is computed on the discounted amount, which is worth stating as the actual business logic being modeled, not just a syntax point.
- `def final_price(price, tier, tax_rate):` — a **new** function whose entire body is two calls to the *other* two functions, chained together. This is composition made explicit: `final_price` doesn't duplicate any discount or tax logic itself — it delegates both jobs out and combines the results.
- `discounted = apply_discount(price, tier)` then `return discounted + calculate_tax(discounted, tax_rate)` — one function's return value (`discounted`) becomes the input to a second function call, inside a third function's own definition. If this feels like a lot of layers, that's worth naming explicitly: composition is genuinely one of the more abstract ideas in the whole course so far, and it's fine if it takes the room a moment to track all three functions at once.

Run it. Expected output:

```
After gold discount: $170.00
After tax: $181.90
final_price(200, "gold", 0.07) = $181.90
```

Common student mistakes to watch for:

- Passing the *original* price (`200`) instead of the discounted price (`170.00`) into `calculate_tax` — computes tax on the wrong base amount, producing `$14.00` tax instead of `$11.90`, and a final total of `$184.00` instead of the correct `$181.90`. A good "does this number look plausible" check: ask the room to sanity-check the final answer against the two inputs before accepting it.
- Defining `final_price` but forgetting that it depends on `apply_discount` and `calculate_tax` already being defined *earlier in the file* — if a student reorders their script and calls `final_price` before those two functions are defined, they'll get a `NameError`. Worth a brief note: Python reads and defines things top to bottom, so a function has to be defined before (or at least by the time) it's called, not necessarily before it's *written* elsewhere later in a more complex program with different call patterns — but for this lab's straightforward top-to-bottom script, "define before you call" is the practical rule.

Check for understanding: "How many separate function *calls* happen when I run `final_price(200, "gold", 0.07)` once — count them all, including calls inside calls." (Three: `final_price` itself, plus its own call to `apply_discount`, plus its own call to `calculate_tax` — getting a student to count all three, not just the outer one, confirms they're tracking the composition correctly.)

6.5 Exercise 4 — Loop + Function (0:26–0:34, 8 min)

Teaching goal: Call `final_price` repeatedly over a list of `(price, tier)` pairs — Module 06's loop skills combined with today's functions, producing a genuine multi-row report from very little code.

Say to the class:

> *"Four orders, four different price/tier combinations — one loop, one function call inside it, and a complete report."*

Live-code this:

```
# --- Exercise 4 ---
orders = [(200, "gold"), (150, "silver"), (80, "bronze"), (500, "none")]
for price, tier in orders:
    total = final_price(price, tier, 0.07)
    print(f"Price: ${price} | Tier: {tier} | Final: ${total:.2f}")
```

Line-by-line explanation:

- `orders = [(200, "gold"), (150, "silver"), (80, "bronze"), (500, "none")]` — a list of **tuples**, each holding two related values (a price and a tier) as one unit. This is the first time this semester a list holds anything other than plain numbers or strings — flag it explicitly as a new, useful shape: "a list of records," where each record bundles multiple related fields.
- `for price, tier in orders:` — this **unpacks** each tuple as the loop pulls it out, in one step: on the first pass, `price` is `200` and `tier` is `"gold"`, both bound at once from the first tuple. Contrast this with Module 06's `for sale in sales:`, which only ever bound one variable per pass — this loop binds two, because each list element is itself a two-part tuple.
- `total = final_price(price, tier, 0.07)` — Exercise 3's composed function, called once per order, with the tax rate fixed at `0.07` for every order in this exercise.

Run it. Expected output:

```
Price: $200 | Tier: gold | Final: $181.90
Price: $150 | Tier: silver | Final: $144.45
Price: $80 | Tier: bronze | Final: $81.32
Price: $500 | Tier: none | Final: $535.00
```

Common student mistakes to watch for:

- Writing `for order in orders:` (a single loop variable) and then trying to use `order[0]` / `order[1]` instead of the cleaner unpacked `price, tier` form — not wrong, but a good moment to show both side by side and let the room see why the unpacked version reads more clearly, echoing Module 01's tuple-unpacking preview.
- Forgetting that `final_price` needs all three arguments (`price`, `tier`, *and* `tax_rate`) — a

student who writes `final_price(price, tier)` alone gets a `TypeError: final_price() missing 1 required positional argument: 'tax_rate'`, a good, very readable error message worth reading aloud together if it comes up.

Check for understanding: “If I wanted a different tax rate for a specific order — say the `'none'` tier order should be taxed at 9% instead of 7% — what’s the smallest change to make?” (Replace the fixed `0.07` with a per-order value — e.g., make `orders` a list of three-element tuples `(price, tier, tax_rate)` instead of two, and unpack all three in the loop. Getting a student to propose this extension confirms they understand the loop-plus-function structure well enough to extend it, not just read it.)

6.6 Exercise 5 — Debug First, Then Ask (0:34–0:54, 20 min)

Teaching goal: A structured, four-step debugging discipline — solo tracing, then a narrowly-scoped AI question, then a self-written fix, then a class debrief. This is the lab's most important exercise for long-term habits, independent of any single Python concept.

Say to the class, before starting the timer:

> *"This is different from everything else today. You're going to debug a broken file called `broken_sales.py` in four timed, five-minute steps. The goal isn't just fixing the bugs — it's building the* habit *of debugging methodically before reaching for an AI tool, and using AI narrowly and honestly when you do reach for it. I will call time at each five-minute mark — don't rush ahead to the next step early, and don't fall behind either."*

Distribute or project `broken_sales.py` (a verified three-bug version is in Appendix B if your course doesn't already provide one — one `SyntaxError`, one silent logic bug, one `IndexError`, deliberately spanning three different bug categories so students practice recognizing each kind).

6.6.1 Step 1 — Solo Debug (5 min)

Say to the class:

> *"Five minutes, alone. Add `print()` statements to trace variable values. Read whatever traceback you get, carefully, start to finish. Then — genuinely — explain the suspect section out loud to a rubber duck, your water bottle, your phone, anything patient. Write down what you think each bug is* before *you fix anything."*

Facilitation notes:

- This is **rubber duck debugging**, from *The Pragmatic Programmer* (Hunt & Thomas, 1999) — mention the source explicitly, it's a real, well-established technique, not a classroom gimmick, and naming that seems to make students take the "talk to an object" instruction more seriously rather than less.
- Circulate during this step specifically to catch students who try to fix bugs immediately without writing down a diagnosis first — the written diagnosis-before-fix requirement is what makes this step teach something, rather than just being five minutes of trial and error.
- Expect the room to hit the bugs in a specific forced order: the file will not run *at all* until the `SyntaxError` (missing colon) is fixed, so every student's first five minutes will center on that one, regardless of how many bugs exist — this is fine and expected; the remaining two bugs surface only after this one is resolved, likely in Step 3, not Step 1.

6.6.2 Step 2 — AI Explainer (5 min)

Say to the class:

> *"Now, paste* only the error message *into an AI assistant — not your code, just the error text. Ask 'what does this error mean?' Do not ask it to fix your code. This step is about understanding the error category, not outsourcing the fix."*

Facilitation notes:

- The "paste only the error message, not the code" constraint is the entire pedagogical point of this step — it forces the AI's response to be about the *general meaning* of the error type, not a specific patched version of this student's file. Watch for students pasting their whole file anyway out of habit; redirect them back to the message-only version if you see it.
- State your own policy explicitly here (which tool is permitted, whether this counts under your syllabus's AI-use policy) — this is exactly the kind of moment reflection question 3 asks students to write an honest disclosure about, so model that honesty yourself by being clear about the rules in the moment.

6.6.3 Step 3 — Fix It Yourself (5 min)

Say to the class:

> *"Use the AI's explanation as a guide, not a script. Write your fix, in your own words, as a comment — before you actually implement it. Then implement it and confirm the file runs correctly."*

Facilitation notes:

- The comment-before-implementation requirement is a forcing function: a student who can't articulate the fix in their own words probably doesn't understand it yet, even if they know which line to change. This is worth stating explicitly if a student tries to skip straight to editing code.
- By this point, most students should be encountering the second bug (the logic bug — total silently computing as `$90` instead of the correct `$715`) and possibly the third (`IndexError`) — if a student is still stuck purely on the syntax error at this point, that's useful information for you about where they need more support, worth a quick individual check-in rather than a whole-class pause.

6.6.4 Step 4 — Class Debrief (5 min)

Say to the class:

> *"Share what you found. Did the AI's explanation match what the bug actually turned out to be?"*

Facilitation notes:

- Cold-call 2–3 students, specifically asking them to name **which of the three bugs** they're

describing (syntax, logic, or runtime/index) — this reinforces the categorization skill, not just "I fixed something."

- A genuinely good discussion question to pose to the whole room: "for which of the three bugs was the AI's explanation most useful, and for which was your own tracing more useful?" Different students will likely have different answers depending on which bug they reached in which step — that variation is itself worth surfacing, since it undercuts any single "AI is always/never the right tool" narrative in favor of a more honest, situational one.
- If time allows, ask explicitly: "which bug would you have found fastest *without* any AI step at all?" Most rooms will say the logic bug (the wrong total) — since noticing "$90 doesn't look like a sum of five positive numbers" is a human sanity-check, not something an error message would have caught, since it never raised an error in the first place. This is worth naming as the exercise's real thesis: **AI is well-suited to explaining error messages you don't understand; it is not a substitute for the sanity-checking instinct that catches errors nothing ever raised.**

6.7 Exercise 6 — Scope Experiment (0:54–1:02, 8 min)

Teaching goal: Local vs. global scope — a genuinely surprising result the first time you see it, and essential for understanding why functions don't accidentally corrupt variables outside themselves.

Say to the class:

> *"Predict the output of this before I run it — write down your three guesses."*

Live-code this (after collecting predictions):

```
# --- Exercise 6 ---
x = 100   # global variable

def double_it():
    x = 999   # local variable — different from global x!
    return x * 2

print(x)             # what prints here?
print(double_it())   # what prints here?
print(x)              # has x changed?
```

Run it. Expected output:

```
100
1998
100
```

Line-by-line explanation, in the order the room needs to reconcile with their predictions:

- `x = 100` — a **global** variable: defined outside any function, at the top level of the script.
- `print(x)` (first) — straightforwardly prints `100`, the global value. No surprises yet.
- Inside `double_it()`, `x = 999` — this creates a **brand new, separate local variable**, also named `x`, that exists *only* inside this function call, and only for its duration. This is the crucial, non-obvious fact: **assigning to `x` inside a function does not modify the global `x`** — it creates a completely independent variable that just happens to share the same name. Say this twice, differently: "there are now two `x`s that both exist briefly at the same time — a global one worth `100`, and a local one worth `999` — and the function has no way to see or touch the global one once it's created its own local `x`."
- `return x * 2` — inside the function, `x` unambiguously refers to the *local* `999` (Python always looks for the nearest, most local version of a name first), so this returns `1998`.
- `print(x)` (third, after the function call) — back at the global level, the local `x` from inside `double_it()` no longer exists at all — it was destroyed the moment the function returned. The global `x` was never touched, so this prints `100` again, unchanged.

Have every student write, in their own words, a one-sentence explanation of scope, as a

comment in their file — this is the exercise's actual required deliverable.

Common student mistakes to watch for:

- Predicting the third `print(x)` will show `999`, assuming the function "changed" the global variable — this is the single most common misconception this exercise exists to correct; if most of the room predicted this, spend extra time on the "two separate `x`s" framing rather than moving on quickly.
- Confusing this with the *opposite*, equally real situation: a function *reading* a global variable (without reassigning it) genuinely can see the global value directly. If a curious student asks "so can a function ever see a global variable at all?", the honest answer is yes — reading is fine; it's specifically *assignment* inside a function that creates a new local variable instead of modifying the global one. This nuance is intentionally outside this exercise's required scope, but worth a truthful one-sentence answer if asked rather than a simplified but wrong one.

Check for understanding: "If I renamed the local variable inside `double_it()` from `x` to something else, like `y`, would the function's behavior change at all?" (No — `double_it()` would still work identically, since it never actually needed to reference the global `x`; the naming collision in the original version was deliberately confusing on purpose, to make the scope lesson vivid, not because the function requires that specific name.)

6.8 Stretch 1 — Summarize a List with a Function (1:02–1:10, 8 min)

Teaching goal: Package Module 06's entire accumulator-pattern lab into a single reusable function that returns a **dictionary** — a new data structure, holding several named results from one function call.

Say to the class:

> *"Everything from last module's sum/count/average/max exercises, wrapped into one function that returns all four results — plus a minimum — bundled together in a dictionary."*

Live-code this:

```
# --- Stretch 1 ---
def summarize(sales_list):
    total = 0
    count = 0
    current_max = sales_list[0]
    current_min = sales_list[0]
    for sale in sales_list:
        total += sale
        count += 1
        if sale > current_max:
            current_max = sale
        if sale < current_min:
            current_min = sale
    return {
        "total": total,
        "count": count,
        "average": total / count,
        "max": current_max,
        "min": current_min,
    }

result = summarize([120, 80, 250, 175, 90, 410, 60, 215])
for key, value in result.items():
    print(f"{key}: {value}")
```

Line-by-line explanation:

- `current_max = sales_list[0]` and `current_min = sales_list[0]` — initialized to the list's **first actual element**, not a hardcoded `0` — recall Module 06 Exercise 3's flagged limitation: initializing to `0` breaks if the list could contain values below zero. This function is written to be genuinely correct for any list, including one with negative numbers, which is worth calling

out as the “more robust” version promised back in Module 06.

- The loop body runs all four accumulator updates (`total`, `count`, `current_max`, `current_min`) together, in one pass through the list — this is the exercise’s real point: one loop can maintain several accumulators simultaneously, not just one.
- `return {"total": total, "count": count, ...}` — a **dictionary literal**: curly braces, `"key": value` pairs separated by commas. This is the first time this semester a function returns something other than a single number or string — it’s returning a small bundle of *named* results at once, which the caller can then access by key.
- `for key, value in result.items():` — `.items()` gives back each key/value pair together, unpacked into two loop variables in one step, the same unpacking idea as Exercise 4’s `for price, tier in orders:`.

Run it. Expected output:

```
total: 1400
count: 8
average: 175.0
max: 410
min: 60
```

Common student mistakes to watch for:

- Initializing `current_max`/`current_min` to `0` out of habit from Module 06’s original version, rather than `sales_list[0]` — harmless for this specific all-positive dataset, but worth pointing out explicitly as the exact limitation flagged back in Module 06, now finally worth fixing properly since this function is meant to be reusable on data the author doesn’t control in advance.
- Forgetting the trailing comma after the last dictionary entry (`"min": current_min` with no comma before the closing `}`) — actually valid Python (a trailing comma is optional, not required), but if a student *removes* a comma between two entries by mistake instead, that’s a real `SyntaxError` worth reading together if it comes up.

Check for understanding: “If I call `summarize([])` — an empty list — what happens?” (`sales_list[0]` raises `IndexError: list index out of range`, since there’s no first element to initialize `current_max`/`current_min` from — a good moment to note that this function, while more robust than Module 06’s original, still isn’t bulletproof against every possible input; handling an empty list gracefully would need an explicit check, which is a good “what would you add” discussion prompt if time allows.)

6.9 Stretch 2 Preview — Default Parameters (as time allows)

Frame as a quick demo if time is short:

```python
def calculate_tax(price, rate=0.07):
    """Return the tax owed on a price at a given rate (default 7%)."""
    return price * rate

print(calculate_tax(100))        # uses the default rate
print(calculate_tax(100, 0.1))   # overrides it
```

Two things worth saying explicitly if you demo this live:

- `rate=0.07` in the parameter list gives `rate` a **default value**, used only when the caller doesn't supply one — `calculate_tax(100)` uses `0.07` automatically; `calculate_tax(100, 0.1)` overrides it with `0.1`.
- **A genuinely worthwhile "gotcha" if you have two spare minutes:** run `print(calculate_tax(100))` and look closely at the output — it's `7.000000000000001`, not exactly `7.0`. This is a real floating-point precision quirk (`100 * 0.07` cannot be represented exactly in binary floating point), not a bug in the function. It's worth demonstrating `100 * 0.07 == 7.0` directly in the REPL and showing it evaluates to `False` — a genuinely useful, real-world caveat about comparing floats for exact equality, and a good preview that "the math is right, but the display isn't always exactly what you'd write by hand" is a recurring theme when working with floats in any language, not just Python.
- The triple-quoted string immediately under `def` is a **docstring** — a description of what the function does, callable later with `help(calculate_tax)` or visible in most editors' hover tooltips. Mention this is genuinely how professional Python code documents functions, not a classroom-only convention.

7 Wrap-Up (last ~5 minutes)

Review the reflection questions out loud:

1. *Which bug did you spot first in Exercise 5, and how?* — Encourage total honesty; there's no "right" order to have found the three bugs in, and the point of the question is building self-awareness about debugging instincts, not demonstrating a particular skill level.
2. *Explain `apply_discount(price, tier)` without looking at the code — could a colleague use it without reading the body?* — A strong answer describes the function's *interface* (what it takes in, what it returns) without describing its internal `if`/`elif` logic at all — this is the real, professional skill of writing functions with clear enough names and behavior that colleagues don't need to read the implementation to use them correctly.
3. *AI disclosure comment, and whether it built or shortcut understanding* — there's no wrong answer, but push for genuine reflection rather than a rote "AI helped me learn" — a strong answer names something specific (e.g., "the AI's explanation of the `IndexError` made me understand why <= was wrong in a way the traceback alone hadn't," or conversely, "I asked for a fix directly on the logic bug and didn't actually understand why my original code was wrong before pasting in a replacement").

Review the submission checklist together:

- ☐ File is named `functions.py`
- ☐ Contains all six required exercises' functions and calls
- ☐ Every function uses `return`, not `print`, for its actual result (except where a script-level `print()` is displaying that result)
- ☐ Exercise 6's scope explanation is written as a comment, in the student's own words
- ☐ Canvas reflection submitted alongside the file

Preview Module 08: "Today you wrote genuinely substantial code — six functions' worth. Next module has zero new Python syntax; instead, you learn Git and GitHub, the version-control workflow every remaining assignment this semester will use to submit your work, and that most software teams use professionally."

8 Appendix A — Full Answer Key (`functions.py`)

```python
# functions.py
# ISM2411 Module 07 Lab — Functions + Debug First, Then Ask

# --- Exercise 1 ---
def calculate_tax(price, rate):
    return price * rate
```

```
print(f"Tax on $100 at 7%: ${calculate_tax(100, 0.07):.2f}")
print(f"Tax on $250 at 8%: ${calculate_tax(250, 0.08):.2f}")

# --- Exercise 2 ---
def apply_discount(price, tier):
    if tier == "gold":
        discount = 0.15
    elif tier == "silver":
        discount = 0.10
    elif tier == "bronze":
        discount = 0.05
    else:
        discount = 0
    return price * (1 - discount)

for tier in ["gold", "silver", "bronze", "none"]:
    print(f"{tier}: ${apply_discount(200, tier):.2f}")

# --- Exercise 3 ---
def final_price(price, tier, tax_rate):
    discounted = apply_discount(price, tier)
    return discounted + calculate_tax(discounted, tax_rate)

after_discount = apply_discount(200, "gold")
after_tax = after_discount + calculate_tax(after_discount, 0.07)
print(f"After gold discount: ${after_discount:.2f}")
print(f"After tax: ${after_tax:.2f}")
print(f'final_price(200, "gold", 0.07) = ${final_price(200, "gold", 0.07):.2f}')

# --- Exercise 4 ---
orders = [(200, "gold"), (150, "silver"), (80, "bronze"), (500, "none")]
for price, tier in orders:
    total = final_price(price, tier, 0.07)
    print(f"Price: ${price} | Tier: {tier} | Final: ${total:.2f}")

# --- Exercise 5: see broken_sales.py (Appendix B) — fixed inline in that file ---

# --- Exercise 6 ---
x = 100   # global variable

def double_it():
```

```
    x = 999   # local variable — separate from global x
    return x * 2

print(x)
print(double_it())
print(x)
# Scope, in my own words: a variable assigned inside a function is local
# to that function and does not affect a same-named variable outside it.
```

Stretch 1 (`Summarize a list with a function`):

```
def summarize(sales_list):
    total = 0
    count = 0
    current_max = sales_list[0]
    current_min = sales_list[0]
    for sale in sales_list:
        total += sale
        count += 1
        if sale > current_max:
            current_max = sale
        if sale < current_min:
            current_min = sale
    return {
        "total": total,
        "count": count,
        "average": total / count,
        "max": current_max,
        "min": current_min,
    }

result = summarize([120, 80, 250, 175, 90, 410, 60, 215])
for key, value in result.items():
    print(f"{key}: {value}")
```

Stretch 2 (`Default parameters`):

```
def calculate_tax(price, rate=0.07):
    """Return the tax owed on a price at a given rate (default 7%)."""
    return price * rate

print(calculate_tax(100))       # 7.000000000000001 — float precision, not a bug
```

```
print(calculate_tax(100, 0.1))  # 10.0
```

9 Appendix B — `broken_sales.py` (for Exercise 5)

A verified, three-bug version to distribute if your course doesn't already provide one on Canvas. Each bug is a distinct category, deliberately: the file **will not run at all** until the first bug is fixed (a `SyntaxError` blocks the whole file from parsing), which forces every student through the same first discovery regardless of where they start looking.

As distributed to students (broken):

```
# broken_sales.py — trace and diagnose before fixing anything.
sales = [120, 80, 250, 175, 90]

total = 0
for sale in sales:
    total = sale
print(f"Total: ${total}")

count = 0
for sale in sales
    count += 1
print(f"Count: {count}")

i = 0
while i <= len(sales):
    print(sales[i])
    i += 1
```

Bug 1 (blocks everything — `SyntaxError`): the `for sale in sales` line on the `count` block is missing its trailing colon. Produces `SyntaxError: expected ':'`, and because Python parses the *entire file* before running any of it, nothing above or below this line executes until it's fixed — the very first thing every student sees, regardless of which bug they'd otherwise have found first.

Bug 2 (silent logic bug, no error at all): `total = sale` inside the first loop should be `total += sale`. Once Bug 1 is fixed, the script runs cleanly and prints `Total: $90` — plausible-looking output that is simply wrong (the correct sum is `$715`); nothing about the program signals a problem. This is the bug most students should catch only by sanity-checking the number against a rough mental estimate, not from any error message.

Bug 3 (`IndexError`, off-by-one): `while i <= len(sales):` should be `while i < len(sales):`. `len(sales)` is `5`, and valid indices are `0` through `4` — the `<=` comparison lets the loop attempt `sales[5]`, which doesn't exist, raising `IndexError: list index out of range` on the sixth pass.

Fully corrected version:

```
# broken_sales.py — fixed
sales = [120, 80, 250, 175, 90]

total = 0
for sale in sales:
    total += sale
print(f"Total: ${total}")

count = 0
for sale in sales:
    count += 1
print(f"Count: {count}")

i = 0
while i < len(sales):
    print(sales[i])
    i += 1
```

Verified output of the fixed version: `Total: $715`, `Count: 5`, then the five sale values printed one per line.

10 Appendix C — Extra Practice (only if the class finishes early)

Six required exercises plus Exercise 5's full 20-minute protocol and Stretch 1 already fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — one more composed function. Have students write `def shipping_cost(order_total): return 0 if order_total >= 75 else (3.99 if order_total >= 25 else 6.99)` (or, more readably, the equivalent `if`/`elif`/`else` form) and a second function `def order_total_with_shipping(order_total): return order_total + shipping_cost(order_total)`. Test with `order_total = 20`, `50`, `100`. (Totals: `$26.99`, `$53.99`, `$100.00`.)

Extra — a second scope trace. Have students predict, then verify:

```
count = 0
def increment():
    count = count + 1   # what happens here?
    return count
print(increment())
```

This one is a genuinely good escalation from Exercise 6: it raises `UnboundLocalError: cannot access local variable 'count' where it is not associated with a value` — because

Python sees the assignment `count = count + 1` anywhere in the function body and treats `count` as local *for the entire function*, which means the right-hand side's *read* of `count` is trying to read a local variable that doesn't have a value yet, even though a global `count` exists. This is a more advanced scope subtlety than Exercise 6 requires, but a genuinely rewarding one for a room that finished early and wants to be pushed further.